Overview

Linear algebra is foundational to AI engineering because modern models process data as vectors and matrices. Features, embeddings, activations, and model weights are all represented in linear algebra structures.

In practical terms: - A text embedding is a vector. - A mini-batch of embeddings is a matrix. - Neural network layers are matrix multiplications plus non-linearities.

If you understand vector and matrix operations, you can reason about ML pipelines, retrieval systems, and neural network behavior with much more clarity.

Core Concepts

Scalars, Vectors, Matrices, Tensors

- **Scalar**: A single number (e.g., temperature = 32.5).
- **Vector**: Ordered list of numbers (e.g., user preference embedding).
- **Matrix**: 2D table of numbers (e.g., batch of embeddings).
- **Tensor**: Generalized n-dimensional array (e.g., image batches).

Matrix Addition, Multiplication, and Transpose

- **Addition** requires equal shapes.
- **Scalar multiplication** scales every element.
- **Matrix multiplication** combines linear transformations when inner dimensions match.
- **Transpose** swaps rows and columns and is central in many derivations.

Dot Product, Norm, and Cosine Similarity

- **Dot product** measures directional alignment and weighted interaction.
- **Norm** measures vector magnitude.
- **Cosine similarity** compares angle between vectors (scale-invariant directional similarity).

Identity Matrix and Inverse (Conceptual)

- **Identity matrix (I)** leaves vectors unchanged when multiplied.
- **Inverse (A^-1)** reverses a matrix transformation when it exists.
- In real ML systems, inverse may be unstable or unavailable; pseudo-inverse and regularization are often used instead.

Linear Algebra in ML/AI

Example: Linear Regression as Matrix Multiplication

Linear regression can be written compactly as:

- `y_hat = X * theta`
- `theta = (X^T X)^-1 X^T y` (normal equation)

This formulation shows that model fitting and prediction are linear algebra operations.

Example: Embeddings and Cosine Similarity

In recommendation and semantic search, each item/document/query is mapped to an embedding vector. Similarity scoring is often cosine-based:

- Higher cosine similarity -> more semantically aligned vectors.

Example: Matrices in Neural Networks

Neural layers apply operations like:

- `z = XW + b`

Where `X` is input activations, `W` is weight matrix, and `b` is bias vector.

Common Pitfalls

- **Shape mismatches**: incorrect dimensions in matrix multiplication.
- **Numerical stability issues**: near-singular matrices and floating-point precision limitations.
- **Scaling problems**: poorly scaled features can distort optimization and similarity behavior.
- **Similarity vs distance confusion**: cosine similarity and Euclidean distance answer different questions.

Business Use Cases

Linear algebra underpins multiple production AI applications:

- **Recommendation systems**: user/item embeddings, similarity ranking.
- **Document similarity and search**: retrieval in semantic search and RAG pipelines.
- **Computer vision**: image tensors, convolution operations, projection layers.
- **Fraud/risk modeling**: vectorized feature pipelines and linear model baselines.

Interview Prep Checklist

- Explain vector vs matrix with practical ML examples.
- Compute and interpret a dot product.
- Explain norm and why normalization matters.
- Explain cosine similarity and where it is preferred.
- Describe shape rules for matrix multiplication.
- Explain linear regression in matrix form.
- Describe where matrices appear in neural networks.
- Explain why matrix inverse can be unstable in practice.
- Distinguish similarity metrics from distance metrics.
- Describe one production system where linear algebra is central.